

MODEL VALIDATION, OVERFITTING CONTROL & HYPERPARAMETER TUNING

Artificial Intelligence & Machine Learning Internship – Task 3

Project: House Price Prediction using California Housing Dataset

Prepared By: Mohit Rajak

B.Tech Computer Science & Engineering

This report presents a complete workflow for improving a Decision Tree based house price prediction model through model validation, overfitting analysis, cross-validation, and hyperparameter tuning. The goal of this task was not only to train a predictive model, but also to evaluate its reliability and generalization performance on unseen data.

1. Introduction

House price prediction is a common regression problem in machine learning where the objective is to estimate the price of a house based on a set of input features such as income, number of rooms, house age, and location. For this task, the California Housing Dataset was used because it provides a realistic multi-feature regression problem and is widely used for benchmarking machine learning models. The central aim of Task 3 was to move beyond simple model training and focus on model quality, stability, and generalization.

A Decision Tree Regressor was selected as the primary model because it can capture non-linear relationships and feature interactions without requiring strict assumptions about the data distribution. However, decision trees can easily become overly complex and memorize training data, which leads to overfitting. To address this, the project was structured around four key ideas: baseline model training, overfitting detection, cross-validation, and hyperparameter tuning using GridSearchCV.

2. Dataset Overview

The California Housing Dataset contains housing-related attributes collected from census blocks in California. Each row represents a district-level observation and the target variable corresponds to the median house value. The dataset includes socio-economic and geographical features such as median income, average number of rooms, average number of bedrooms, population, average occupancy, latitude, and longitude.

Before model training, the dataset was loaded into a Pandas DataFrame and explored using summary statistics and visualization techniques. Correlation analysis, target distribution inspection, and feature-level exploration were performed to understand which variables might be most influential for price prediction. This exploratory step is important because it provides intuition about the problem and helps guide model interpretation later.

3. Methodology

The overall workflow of the project followed a structured machine learning pipeline. First, the dataset was split into training and testing subsets. The training set was used to fit the models, while the testing set was kept separate for final evaluation. A baseline Decision Tree Regressor was trained using default settings. Its training and testing performance were then compared to check whether the model was generalizing well or overfitting the training data.

After baseline evaluation, 5-fold cross-validation was applied. Cross-validation divides the data into multiple folds and evaluates the model several times on different train-validation splits. This provides a more reliable estimate of model performance than relying on a single train-test split, because it reduces the chance that results are influenced by one favorable or unfavorable split.

Finally, hyperparameter tuning was performed using GridSearchCV. Different combinations of Decision Tree parameters such as maximum depth, minimum samples required to split a node, and minimum samples required at a leaf node were tested systematically. The best-performing combination was selected based on validation performance, and the tuned model was then evaluated on the test set.

4. Overfitting Analysis

Overfitting occurs when a model learns the training data too closely, including noise and dataset-specific patterns, instead of learning relationships that generalize to unseen examples. Decision Trees are especially prone to this problem because they can keep splitting until they fit the training data very closely. In practice, an overfit model often shows excellent training performance but noticeably weaker testing performance.

To analyze overfitting, the baseline Decision Tree model was trained and then evaluated on the test set. Additional visual tools such as train-vs-test score comparisons, validation curves, and residual analysis help reveal whether model complexity is helping or harming generalization. In this task, the baseline model produced reasonable results, but the objective was to improve stability and predictive quality by controlling model complexity through tuning.

5. Cross-Validation and Hyperparameter Tuning

Cross-validation plays an important role in model validation because it measures how consistently the model performs across multiple subsets of the data. Rather than trusting one split, the model is trained and validated several times. If the cross-validation scores remain stable, it increases confidence that the model will perform reliably on unseen data. This is particularly useful when tuning tree-based models, because their performance can vary significantly depending on the complexity of the tree.

GridSearchCV was used to automate the hyperparameter tuning process. A grid of candidate values was defined for `max_depth`, `min_samples_split`, and `min_samples_leaf`. The algorithm trained multiple models across these combinations and selected the configuration that produced the strongest validation performance. This approach ensures that model selection is based on evidence rather than guesswork.

Validation curves can also be plotted for parameters such as `max_depth`. These curves show how training and validation scores change as model complexity increases. If the training score rises while the validation score starts dropping, it indicates overfitting. Such analysis is useful for explaining why a tuned model performs better than the baseline model.

6. Results and Performance Comparison

The final performance of the baseline and tuned Decision Tree models was measured using RMSE (Root Mean Squared Error) and R^2 Score. RMSE measures the average magnitude of prediction error, so lower values indicate better predictions. R^2 Score measures how much of the variance in house prices is explained by the model, so higher values indicate stronger predictive performance.

Model	RMSE	R^2 Score
Baseline Decision Tree	0.7037	0.6221
Tuned Decision Tree	0.6391	0.6883

The results clearly show that the tuned Decision Tree model outperformed the baseline model. The RMSE improved from 0.7037 to 0.6391, which indicates a lower prediction error after tuning. Similarly, the R^2 Score increased from 0.6221 to 0.6883, meaning the tuned model was able to explain a larger proportion of the variance in house prices.

This improvement demonstrates that hyperparameter tuning was effective in controlling model complexity and improving generalization. Instead of relying on the default structure of the tree, the tuned model selected a better balance between fitting the training data and maintaining predictive accuracy on unseen test examples.

7. Interpretation of Results

From a practical perspective, the tuned model is preferable because it provides more reliable predictions with lower error. The reduction in RMSE suggests that the predicted prices are closer to the actual values on average, while the higher R^2 Score indicates that the model captures the underlying relationship between housing features and prices more effectively.

This task also highlights an important lesson in machine learning: a model with strong raw flexibility is not always the best model in its default form. Decision Trees can perform very well, but only when their growth is controlled properly. Techniques such as cross-validation, validation curves, and GridSearchCV make the model development process more disciplined and help avoid misleading results caused by overfitting.

8. Visual Analysis Used in the Project

To support model interpretation, multiple visualizations can be included in the notebook and GitHub repository. A correlation heatmap helps understand feature relationships, while an overfitting bar chart compares train and test R^2 scores. Cross-validation plots show score stability across folds, and a validation curve demonstrates how model complexity affects both training and validation performance.

For final model evaluation, an Actual vs Predicted plot helps assess how closely the model's predictions follow the ideal prediction line. A residual plot helps identify whether the errors are randomly distributed or whether systematic patterns remain. Feature importance charts are especially useful for tree-based models because they show which variables contribute most to the predictions.

9. Conclusion

This project successfully demonstrated the importance of model validation and tuning in a machine learning regression task. A baseline Decision Tree Regressor was first trained on the California Housing Dataset and then evaluated for predictive performance. To make the model more reliable, cross-validation and hyperparameter tuning were applied. The tuned model showed a clear improvement over the baseline, achieving lower RMSE and higher R^2 Score.

Overall, Task 3 provided practical experience in identifying overfitting, validating model performance across multiple folds, and improving a model using GridSearchCV. These steps are essential in real-world machine learning because the goal is not just to fit a model, but to build one that generalizes well and makes dependable predictions on new data.

10. Final Conclusion Statement

The tuned Decision Tree model emerged as the best model for this task, improving prediction quality and generalization compared with the baseline model. This project strengthened understanding of the complete workflow of model validation, overfitting control, and hyperparameter optimization—key concepts for building robust and production-ready machine learning systems.